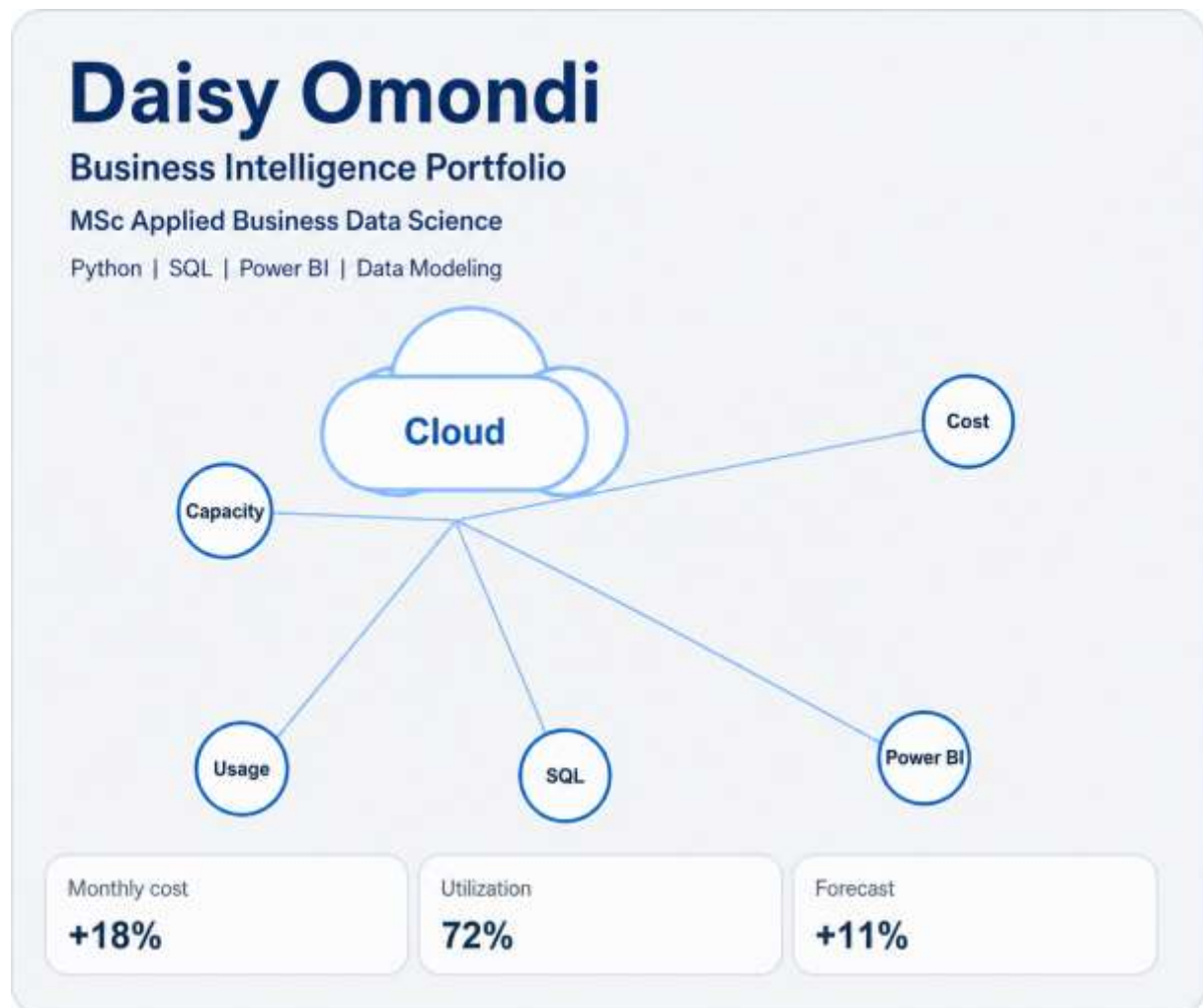


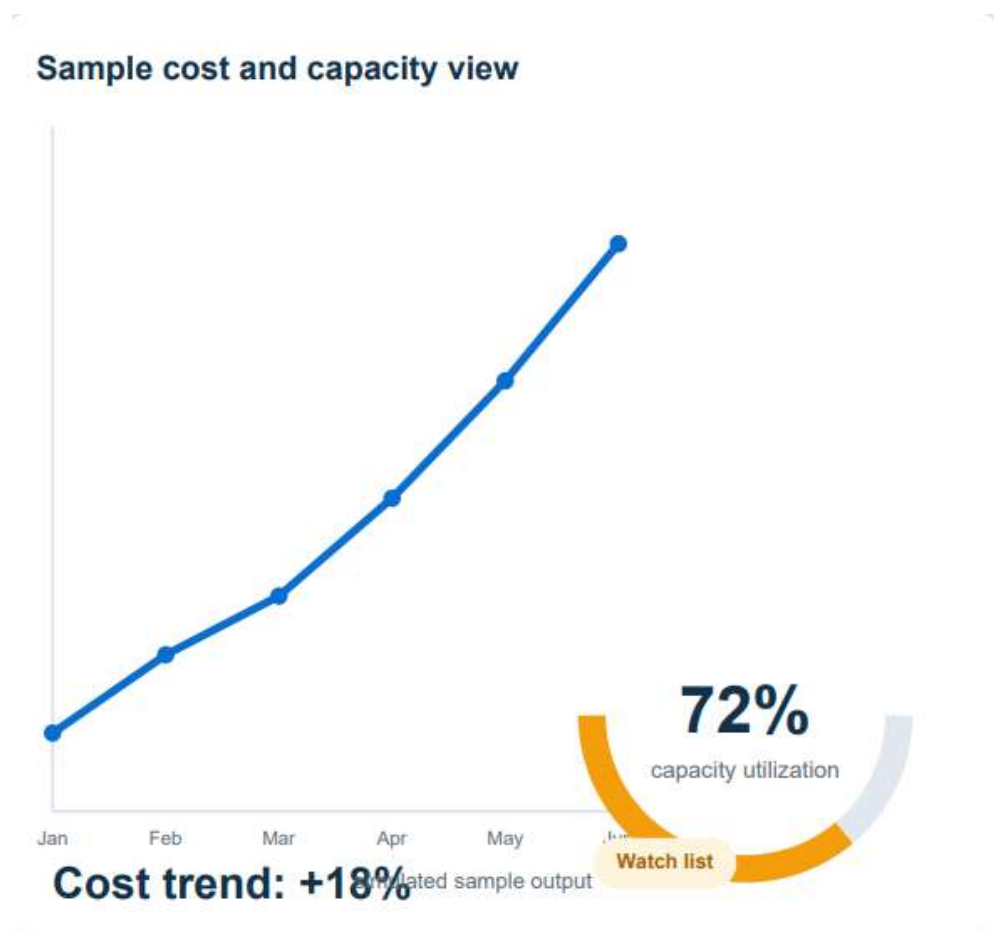
Cloud Capacity & Cost Analytics Pipeline

Turning Cloud Usage Data into Cost Visibility, Capacity Planning and Business Decisions



1. Why I Built This Project

I built this portfolio project to show that an operations problem can be taken outside healthcare and turned into clear business reporting. Cloud usage can grow quickly. If teams only look at costs at the end of the month, they may miss early signs of pressure: services that are getting expensive, regions where demand is growing, or resources that are paid for but not fully used. This project is my way of showing three things: I can clean and structure data, I can analyze cost and capacity patterns, and I can explain the result in a way business users can act on. My healthcare background taught me that data is only useful when it helps people make better decisions earlier. I used that same thinking here, but with cloud capacity and cost data.



2. Data Scope: What I Simulated and Why

The dataset is simulated, but it is structured like data a cloud operations or business analytics team could use. I kept the scope focused. The project is not meant to be a perfect cloud billing system. It is a practical analytics workflow: raw data, cleaning, SQL structure, quality checks, dashboard and recommendations.

Core questions:

- Which services drive the highest monthly cost?
- Where is capacity demand increasing?
- Are there underused resources with high cost?
- Which teams or regions need earlier planning visibility?

usage_records compute hours, storage GB, run time 	services service type and pricing class
regions cloud region and location group 	cost_centers team, department, owner
billing_month monthly cost and forecast period 	quality_log missing, duplicate, inconsistent entries

3. From Raw Data to Business Insight

I also wanted to show that a dashboard is only useful when the data behind it is reliable. For that reason, I did not start the project directly in Power BI. I first focused on the data flow. The raw data needs to be cleaned, checked and structured before it can become useful for reporting.

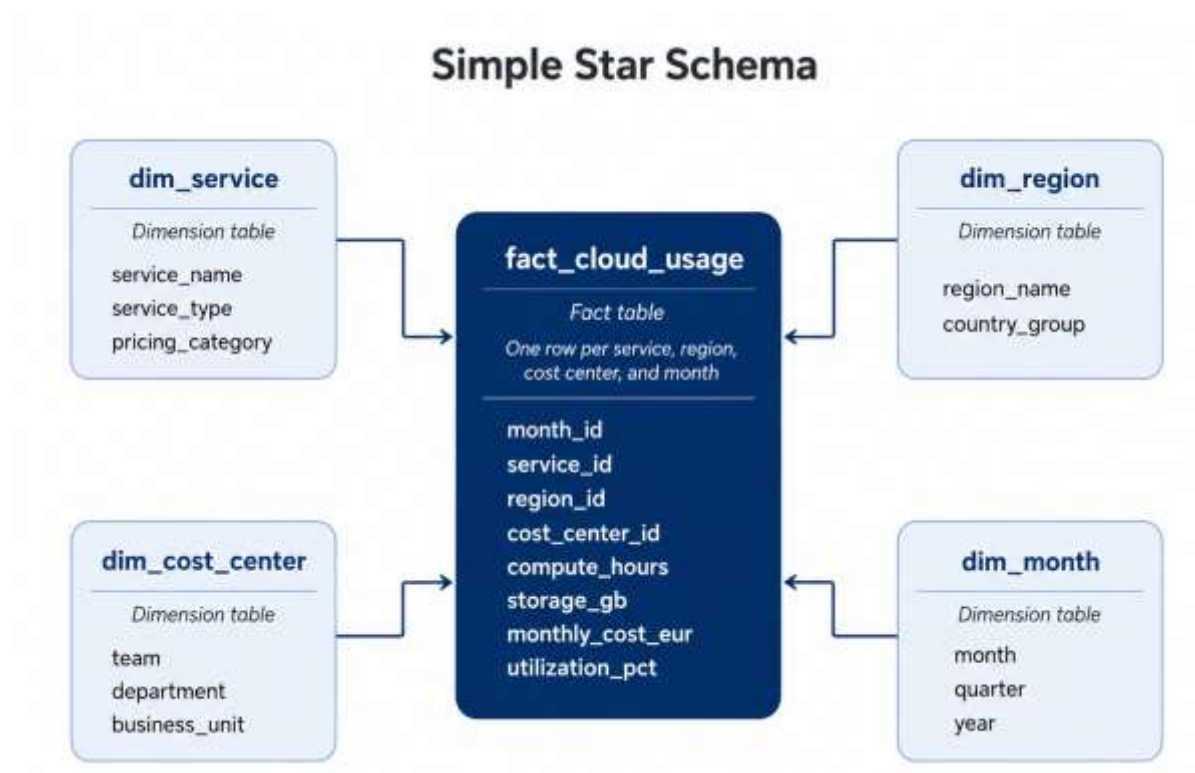
This is important because business users should not have to question every number in a dashboard. If the data is messy, duplicated or inconsistent, the final report may look nice but still lead to wrong decisions. That is why the pipeline follows a clear order: prepare the data, store it in a structured way, check the quality, analyze the patterns and then present the result.

For me, this is also the connection to my future direction in data engineering. I do not only want to build dashboards. I want to understand how data moves from the raw source to a clean structure and finally to a business decision.



4. Data Model, SQL Logic & Quality Checks

I used a simple star schema because it keeps the analysis clean and easy to follow. The central fact table stores the measurable values, such as compute hours, storage usage, monthly cost and utilization. The dimension tables give context to those values, for example which service, region, cost center or month the record belongs to.



This structure makes the project easier to scale than one large flat table. If a business user wants to compare cost by service, region or team, the model already supports that question. It also makes the SQL queries clearer because each table has one clear role.

The quality checks are included because small data issues can quickly create wrong conclusions. For example, a duplicate service-month record could make monthly cost look higher than it really is. A missing region name could make regional reporting incomplete. A negative cost value would not make sense in this context and should be checked before the data is used in a dashboard.

Example SQL Logic & Quality Checks

Example SQL logic:

```
SELECT
    s.service_name,
    r.region_name,
    SUM(f.monthly_cost_eur) AS total_cost,
    AVG(f.utilization_pct) AS avg_utilization,
    SUM(f.compute_hours) AS total_compute_hours
FROM fact_cloud_usage f
JOIN dim_service s
    ON f.service_id = s.service_id
JOIN dim_region r
    ON f.region_id = r.region_id
GROUP BY
    s.service_name,
    r.region_name
HAVING
    SUM(f.monthly_cost_eur) > 5000
    AND AVG(f.utilization_pct) < 40
ORDER BY
    total_cost DESC;
```

This query helps identify cloud services that are expensive but not fully used. These are the services I would place on a cost optimization watch list.

Quality checks included:

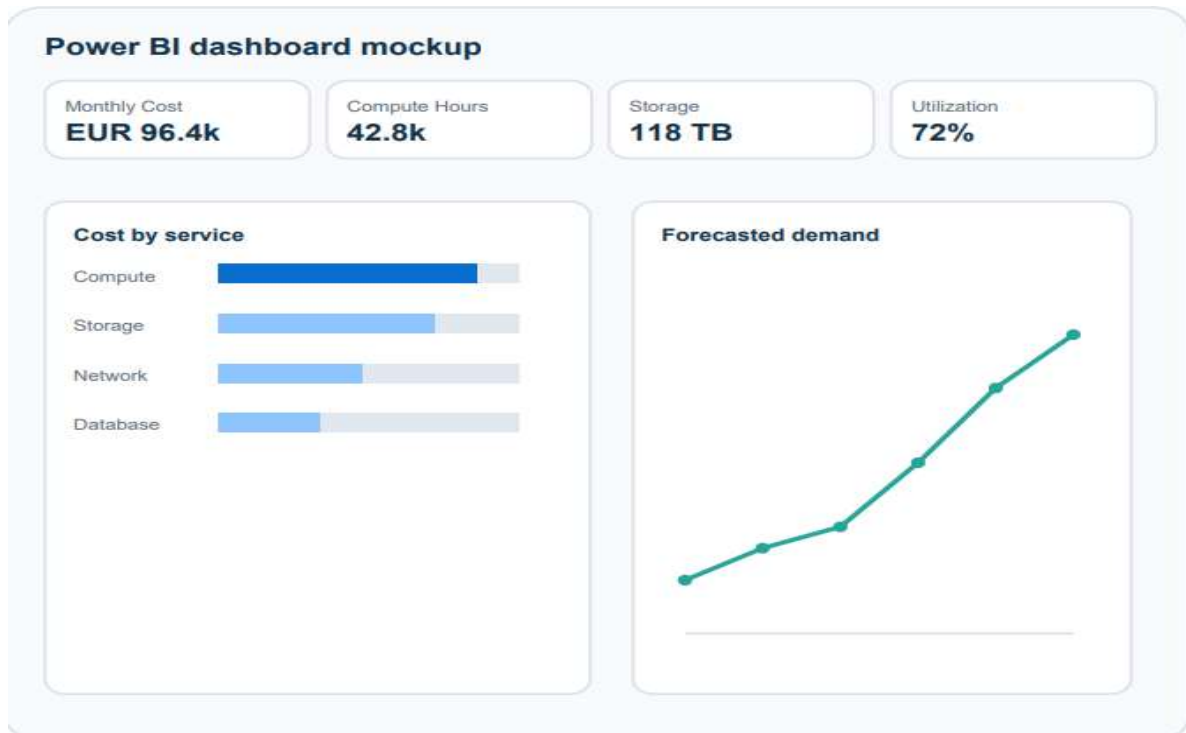
- missing cost or usage values
- duplicate service-month records
- negative or unrealistic cost entries
- inconsistent region names
- utilization values outside expected range

`data_quality_checks.sql`

```
SELECT billing_month, service_id,
    SUM(monthly_cost) AS cost,
    AVG(utilization_rate) AS utilization
FROM fact_cloud_usage
WHERE monthly_cost IS NOT NULL
GROUP BY billing_month, service_id;
```

5. Dashboard for Fast Cost & Capacity Decisions

The dashboard is designed for business users. It should answer the most important questions quickly, without needing to read SQL or Python code. The dashboard focuses on cost, usage and capacity. I kept the design simple because the goal is not to impress with decoration. The goal is to make the right information visible fast.



Main views:

- services where cost increased faster than usage
- compute hour development
- storage usage by service
- early visibility of demand growth
- watch list for expensive or underused services

Key dashboard KPIs:

- Total monthly cloud cost: €48,750
- Cost increase compared to previous month: +18%
- Average capacity utilization: 74%
- Forecasted compute demand next month: +12%
- High-cost services with low utilization: 11 services

These KPIs are based on the simulated dataset. I used them to make the dashboard more practical, because a business user should see the most important information first.

6. What the Numbers Show & What I Would Recommend

The most important part of the project is the ability to turn the numbers into clear recommendations. I want the reader to see how I think: What does the number mean? Why does it matter? What should a team do next? The example insights below are based on the simulated dataset. They show the kind of recommendations I would communicate to a business or operations team.

Example Insights & Recommendations

Example findings from the simulated cloud capacity and cost dataset.

Insight	Meaning	Recommendation
 Compute services made up 42% of monthly cloud cost	Compute is the biggest cost driver	Review the top 3 compute services monthly
 Storage usage in one region increased by 28% over three months	Demand is growing faster in that region	Check whether extra storage is needed or old data can be archived
 11 services had high cost but less than 40% utilization	Some paid resources may be underused	Add them to an optimization watch list
 Forecasted compute demand increased by 12% for next month	Capacity pressure may rise soon	Plan capacity earlier instead of reacting later

The dashboard helps teams see where to ask better questions before cost or capacity pressure becomes a bigger problem. Based on the simulated analysis, I would recommend a monthly cloud cost review, a watch list for low-utilization services, and early planning for regions where demand is growing. I would also separate normal cost growth from possible waste, because not every cost increase is bad. Some cost increases come from real business demand, while others may come from unused or badly planned resources.



7. Why This Project Fits SAP

What makes this project important for me is that it connects my past experience with my next career step. In healthcare, I learned to look at operational pressure early, not only when the problem is already visible. I used the same mindset in this project. Instead of only showing cloud costs after they already happened, the dashboard gives earlier visibility into usage growth, capacity pressure and possible waste.

For SAP, I see this project as a good example of how I approach data work. I start with a practical problem, build a clear data structure, check the quality, analyze the patterns and explain the result in simple language. I know I am still growing technically, but this project shows the direction I am serious about: business analytics first, stronger SQL and data modeling next, and step by step into data pipelines and cloud data engineering.

This is why I see the project as more than a portfolio piece. It shows how I think, how I learn and how I connect business questions with technical work.

